Memento

Memento · Save/restore text editor snapshots · Build with Waqar

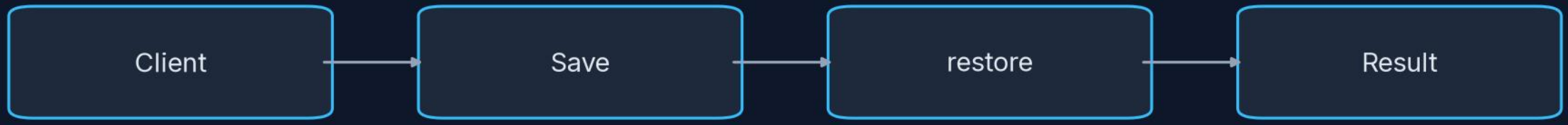
Today's Hook

- Problem: Save/restore text editor snapshots
- Pattern: Memento — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Save/restore text editor snapshots
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Memento: Save/restore text editor snapshots

Real-World Scenario

- Running example: Save/restore text editor snapshots
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.memento.problem;

import java.util.*;

public final class NaiveDoc {
  public String text="";
  public String backup; // single slot only
  public void save(){ backup=text; }
  public void restore(){ text=backup; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.memento.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

final class DocMemento { final String text; DocMemento(String t){ text=t; } }
final class Doc {
  private String text="";
  void write(String s){ text+=s; }
  String text(){ return text; }
  DocMemento save(){ return new DocMemento(text); }
  void restore(DocMemento m){ text=m.text; }
}
final class History {
  private final Deque<DocMemento> stack=new ArrayDeque<>();
  void push(DocMemento m){ stack.push(m); }
  DocMemento pop(){ if(stack.isEmpty()) throw new IllegalStateException("empty"); return stack.pop(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.memento.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Memento: editor snapshots ===");
    System.out.println("--- naive ---");
var n=new NaiveDoc(); n.text="a"; n.save(); n.text="ab"; n.restore(); System.out.println(n.text);
System.out.println("--- memento ---");
var d=new Doc(); var h=new History(); d.write("Hello"); h.push(d.save()); d.write("!");
System.out.println(d.text()); d.restore(h.pop()); System.out.println("restored="+d.text());
  }
}
```

Walkthrough Output

```
=== Memento: editor snapshots ===
--- naive ---
a
--- memento ---
Hello!
restored=Hello
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Memento.

Related Patterns

- Contrast Memento with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Memento → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.